

```
GNU nano 7.2                                                                    fork.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    char command[100];

    // Accept Linux command from user
    printf("Enter a Linux command: ");
    scanf("%s", command);

    // Create a child process
    pid = fork();

    if (pid < 0)
    {
        // Fork failed
        printf("Fork failed!\n");
        exit(1);
    }
    else if (pid == 0)
    {
        // Child Process
        printf("\n==== Child Process =====\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        // Execute the entered command
        execlp(command, command, (char *)NULL);

        // This executes only if exec fails
        printf("Exec failed!\n");
        exit(1);
    }
    else
    {
        // Parent Process
        printf("\n==== Parent Process =====\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        // Wait for child process to finish
        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

```
siri@SIRI CHANDANA:~$ nano fork.c
siri@SIRI CHANDANA:~$ gcc fork_exec.c -o fork_exec
siri@SIRI CHANDANA:~$ ./fork_exec
Enter Linux command: ls

Parent Process
Parent PID = 553

Child Process
Child PID = 556
fork.c    fork_exec.c    helloworld    myShell.c    process    states    task1
fork_exec fork2.c      myShell      practical3.c  process.c   states.c   task1.c
Child process completed
siri@SIRI CHANDANA:~$ nano fork.c
siri@SIRI CHANDANA:~$ gcc fork_exec.c -o fork_exec
siri@SIRI CHANDANA:~$ ./fork_exec
Enter Linux command: pwd

Parent Process
Parent PID = 580

Child Process
Child PID = 581
/home/siri
Child process completed.
siri@SIRI CHANDANA:~$
```

```
satwika@SATWIKAG: × + v
GNU nano 7.2 fork2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    pid_t pid;

    while (1)
    {
        printf("\nEnter Linux Command (or type exit to quit): ");
        fgets(command, sizeof(command), stdin);

        // Remove newline character
        command[strcspn(command, "\n")] = '\0';

        // Exit condition
        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting Program...\n");
            break;
        }

        pid = fork();

        if (pid < 0)
        {
            printf("Fork Failed!\n");
            exit(1);
        }
        else if (pid == 0)
        {
            // Child Process
            printf("\nChild Process Created\n");
            printf("Child PID : %d\n", getpid());
            printf("Parent PID : %d\n", getppid());

            execlp(command, command, (char *)NULL);

            // Executes only if exec fails
            printf("Invalid Command!\n");
            exit(1);
        }
        else
        {
            // Parent Process
            wait(NULL);
            printf("Command Execution Completed.\n");
        }
    }

    return 0;
}
```

```
siri@SIRI CHANDANA:~$ nano fork2.c
siri@SIRI CHANDANA:~$ gcc fork2.c -o fork2
siri@SIRI CHANDANA:~$ ./fork2
Enter Linux Command (or type exit to quit): date

Child Process Created
Child PID : 671
Parent PID : 666
Thu Aug 20 12:26:56 UTC 2026
Command Execution Completed.

Enter Linux Command (or type exit to quit): pwd

Child Process Created
Child PID : 672
Parent PID : 666
/home/siri
Command Execution Completed.

Enter Linux Command (or type exit to quit): exit
Exiting Program...
siri@SIRI CHANDANA:~$
```